

Université de Sherbrooke
Département d'informatique
IGL501/IGL710 : Méthodes formelles en génie logiciel
Devoir 3
Date de remise : vendredi 7 novembre à 16:00

À remettre avec Turninweb (<http://turnin.dinf.usherbrooke.ca/>)

Questions

Question 1

Raffinez la spécification suivante

[relation.mch](#)

L'ébauche suivante de raffinement est fournie; vous devez la compléter pour obtenir un raffinement.

[relation_ref.ref](#)

Cette ébauche utilise une matrice de booléens pour représenter une relation. Vous devez fournir un invariant de collage qui relie l'état de la machine abstraite à celui de la machine concrète.

Pour vérifier votre solution, vous pouvez utiliser ProB comme suit:

1. Faites un model check sur la spécification relation.mch (menu Verify->Model Check)
2. Sauvegardez le résultat de la vérification (menu Verify->Save State for later refinement check)
3. Ouvrez relation_ref.ref et vérifiez la (menu Verify->Model Check)
4. Vérifiez que relation_ref.ref raffine relation.mch (menu Verify->Trace Refinement Check)

Créer la machine relation_ref_PO.mch pour prouver les obligations de preuve de raffinement; inspirez-vous de l'exemple du raffinement sur les ensembles. Utilisez k=2 pour la vérification; k=3 prend beaucoup de temps.

[Ensemble_R1_PO.mch](#)

Question 2

Raffinez la spécification suivante

[file.mch](#)

L'ébauche suivante de raffinement est fournie; vous devez la compléter pour obtenir un raffinement.

[file_ref.ref](#)

Donner aussi la machine `file_ref_PO.mch` pour prouver les obligations de preuve de raffinement.

Question 3

Complétez la machine abstraite [Factoriel.mch](#) qui contient une seule opération qui retourne le factoriel de `n`.

Donnez une implémentation de cette opération `Factoriel_imp.imp`

Donnez la machine `Factoriel_imp_PO.mch` qui contient les obligations de preuve.

Question 4

Modifiez le programme [Maximum.dfy](#) pour produire un nouveau programme **Maximum-bis.dfy** qui satisfait les exigences suivantes :

- Changez la signature de la fonction `max` pour la suivante
`max(x : int, y : int) : int`
afin qu'elle retourne un entier plutôt qu'un `bool`; cet entier est le maximum de `x` et `y`.
- Changez la signature de la fonction `maximum` pour la suivante
`method Maximum(x1 : int, x2 : int, x3 : int) returns (z : int)`
`ensures z == max(x3, max(x1, x2))`
et modifiez son implémentation en conséquence

Question 5

Modifiez le programme [Somme.dfy](#) pour produire un nouveau programme **Produit.dfy** qui satisfait les exigences suivantes :

- Définissez la méthode suivante

`method Produit(a: array<int>) returns (s : int)`
`ensures s == prod(a, a.Length-1)`
- Comme cette spécification utilise la fonction `prod`, définissez-la afin qu'elle retourne le produit des éléments 0 à `i` du vecteur `a`

`function prod(a: array<int>, i : int): int`

Question 6

Écrivez le programme **Palindrome.dfy**, qui vérifie si le vecteur `a` est un palindrome. Un palindrome est un vecteur qui contient la même suite d'éléments, peu importe si on lit le vecteur du début à fin, ou de la fin au début; par exemple, le mot « ressasser » est un palindrome, de même que la phrase « Élu par cette crapule ». Voici la signature de la méthode de ce programme

method EstPalindrome(a: array<int>) returns (r : bool)

Pour écrire la spécification de cette méthode, écrivez un prédicat avec la signature suivante

predicate palindrome(a: array<int>, i : int)

qui retourne vrai ssi a est un palindrome de la position 0 à i, c'est-à-dire que

$a[0] == a[a.length-1] \ \&\& \ a[1] == a[a.length-2] \ \&\& \ ... \ \&\& \ a[i] == a[a.length-1-i]$